



Atividade Lab: Processamento Distribuído de Imagens Médicas com MPI

Prof. Alcides / Prof. Mário

Aluno: Artur Campi - RA: 10436740

Curso: Ciência da Computação | Turma 06N

Disciplina: Computação Distribuída

Atividade individual | Data de submissão: 27/09/2026

Repositório: github.com/Artur-Campi/lab-mpi-processamento-imagens



Sumário

1. Introdução e Fundamentos Teóricos
2. Metodologia e Arquitetura do Cluster
3. Evidências Experimentais (capturas do terminal do GitHub Codespaces)
4. Tabela Comparativa de Desempenho e Speedup
5. Respostas às 9 Questões de Reflexão
6. Dificuldades Técnicas e Soluções
7. Declaração e Análise do Uso de IA
8. Conclusão e Referências

Todos os códigos, logs brutos das execuções, o CSV com as 108 medições, a tabela gerada e as capturas de tela em resolução original estão no repositório público: <https://github.com/Artur-Campi/lab-mpi-processamento-imagens>.

1. Introdução e Fundamentos Teóricos

Em períodos de alta demanda hospitalar, redes de saúde precisam triar rapidamente milhares de radiografias de tórax. Uma radiografia digital em tons de cinza é uma matriz $M \times N$ de intensidades entre 0 (regiões escuras, ar) e 255 (regiões radiopacas, ossos e tecidos densos). Áreas anormalmente claras dentro dos campos pulmonares podem indicar infiltrados, consolidações ou opacidades. Como a análise de cada pixel é independente das demais, o problema é um caso clássico de **paralelismo de dados**: a mesma operação é aplicada a partes disjuntas da entrada, e os resultados parciais são combinados no final (padrão SPMD, Single Program Multiple Data).

O **MPI (Message Passing Interface)** é o padrão de fato para programação distribuída em memória distribuída: cada processo possui seu próprio espaço de endereçamento e só troca dados por mensagens. Além da comunicação ponto a ponto, o MPI define **operações coletivas**, executadas por todos os processos de um comunicador (aqui, `COMM_WORLD`). As cinco usadas neste laboratório são:

- **MPI_Bcast** (`comm.bcast`): um processo raiz envia o mesmo dado a todos. Ideal para parâmetros pequenos e globais.
- **MPI_Scatter** (`comm.scatter`): o raiz divide um conjunto em `size` partes e entrega uma parte a cada processo (inclusive a si mesmo).
- **MPI_Barrier** (`comm.Barrier`): nenhum processo avança até que todos tenham chegado ao mesmo ponto.
- **MPI_Reduce** (`comm.reduce`): combina valores de todos os processos com um operador associativo (`MPI.SUM`, `MPI.MAX`) e entrega o resultado ao raiz.
- **MPI_Gather** (`comm.gather`): o raiz recebe uma lista com o objeto individual de cada processo, preservando a identidade de cada um.

Na biblioteca **mpi4py**, os métodos em minúsculas (`bcast`, `scatter`, `reduce`, `gather`) serializam objetos Python arbitrários com `pickle`, o que é conveniente para dicionários e arrays NumPy; os métodos com inicial maiúscula (`Bcast`, `Scatterv...`) operam diretamente sobre buffers contíguos, sem serialização.

Dois limites teóricos guiam a análise dos resultados. A **Lei de Amdahl** afirma que, se uma fração f do trabalho é sequencial, o speedup com p processos é $S(p) = 1 / (f + (1 - f)/p)$, limitado a $1/f$ mesmo com infinitos processos. Já a **granularidade** (razão entre computação e comunicação) determina se vale a pena distribuir: se cada processo calcula pouco em relação ao custo de receber os dados e sincronizar, o overhead domina e o tempo



aumenta. Por fim, em aplicações síncronas (com barreiras e coletivas bloqueantes) o tempo de cada fase é ditado pelo processo mais lento, o chamado **straggler effect**.

2. Metodologia e Arquitetura do Cluster

2.1 Infraestrutura utilizada

Todos os experimentos foram executados em um **GitHub Codespace** criado a partir do repositório do laboratório, com máquina de 4 núcleos. Dentro dele, o Docker (docker-in-docker) sobe um cluster de 4 containers ligados por uma rede bridge dedicada (mpinet). Os dados da tabela abaixo foram coletados no próprio Codespace (resultados/ambiente_codespace.txt e print da Figura 2).

Componente	Especificação
Host (GitHub Codespaces)	4 vCPUs AMD EPYC 7763 (2 núcleos físicos x 2 threads SMT), 15 GiB RAM, Ubuntu 24.04.5 LTS, kernel 6.8.0-azure
Docker / Compose	Docker 29.8.0 / Docker Compose v5.5.1
Imagem dos nós	ubuntu:22.04 (mesma imagem para os 4 containers: lab-mpi:latest)
MPI	Open MPI 4.1.2 (openmpi-bin), lançamento remoto via SSH (usuário mpiuser, chave ed25519 sem senha)
Python e bibliotecas	Python 3.10.12, mpi4py 3.1.3, NumPy 1.21.5 (pacotes apt python3-mpi4py e python3-numpy)
Nós do cluster	master, worker1, worker2, worker3 (hostfile com slots=1 por nó; 8 processos exigem --oversubscribe)
Parâmetros MCA	btl_vader_single_copy_mechanism=none; btl_tcp_if_include=eth0; oob_tcp_if_include=eth0

Observação importante para a interpretação dos tempos: os 4 containers compartilham os mesmos 4 vCPUs do Codespace, que correspondem a apenas **2 núcleos físicos** com hyperthreading. A "rede" entre nós é virtual (bridge Linux), então o custo de comunicação é dominado por serialização (pickle) e cópias de memória, e não pela latência de uma rede física.

2.2 Arquivos do projeto

Arquivo	Função
processamento_imagens.py	Programa MPI com as 12 etapas do roteiro (parametrizável por linha de comando)
Dockerfile / docker-compose.yml / hosts	Imagem com Open MPI + mpi4py + SSH e definição dos 4 nós
preparar_cluster.sh	Sobe o cluster, corrige o iptables do Codespaces, inicia o SSH e copia o script para os nós
executar_experimentos.sh	Bateria de 108 execuções (3 repetições por configuração) gravadas em resultados/tempos.csv
analisar_resultados.py	Calcula médias, desvio, speedup, eficiência e gera resultados/grafico_speedup.png

2.3 Implementação das 12 etapas

O programa segue rigorosamente as etapas do roteiro. Os pontos principais de cada uma:



Etapa	Implementação
1 Inicialização	comm = MPI.COMM_WORLD, rank, size; rank 0 é o root/master
2 Radiografia simulada	Só o root gera a matriz uint8 com fundo escuro, partes moles, mediastino e coluna claros, campos pulmonares escuros com sombra de costelas, diafragma e 3 focos de consolidação (2 no lado direito da matriz, 1 pequeno no esquerdo). Semente fixa (2026), então a imagem é idêntica em todos os testes
3 Bcast	Dicionário com linhas, colunas, limiar_suspeito=200, limiar_alto=230, pct_critico=5%, pct_atencao=1%, pct_alto_critico=1%, atraso e modo de análise
4 Barrier inicial	comm.Barrier() após a parametrização
5 Scatter	np.array_split em faixas contíguas de linhas (tamanhos diferem em no máximo 1) + índice global da 1ª linha de cada faixa
6 Análise local	Total, soma (uint64), máximo, suspeitos (>200), altamente suspeitos (>230), suspeitos por pulmão (coluna < N/2 e >= N/2). Dois modos: vetorizado (NumPy) e laço pixel a pixel em Python puro
7 Classificação da faixa	CRITICA se suspeitos >= 5% ou altos >= 1%; ATENCAO se suspeitos >= 1%; senão NORMAL
8 Heterogeneidade	Ranks ímpares: time.sleep(0.5 x rank) (fator configurável por --atraso)
9 Barrier pré-consolidação	Mede também quanto cada processo esperou na barreira
10 Reduce	SUM: pixels, soma, suspeitos, altos, esquerdo, direito; MAX: intensidade máxima e maior tempo de análise
11 Gather	Dicionário por processo: rank, host, intervalo de linhas, suspeitos, %, máximo local, classificação, espera na barreira
12 Relatório final	Informações gerais, métricas globais, lateralidade, auditoria por faixa e classificação geral (Sem Indícios / Atenção Clínica / Quadro Crítico)

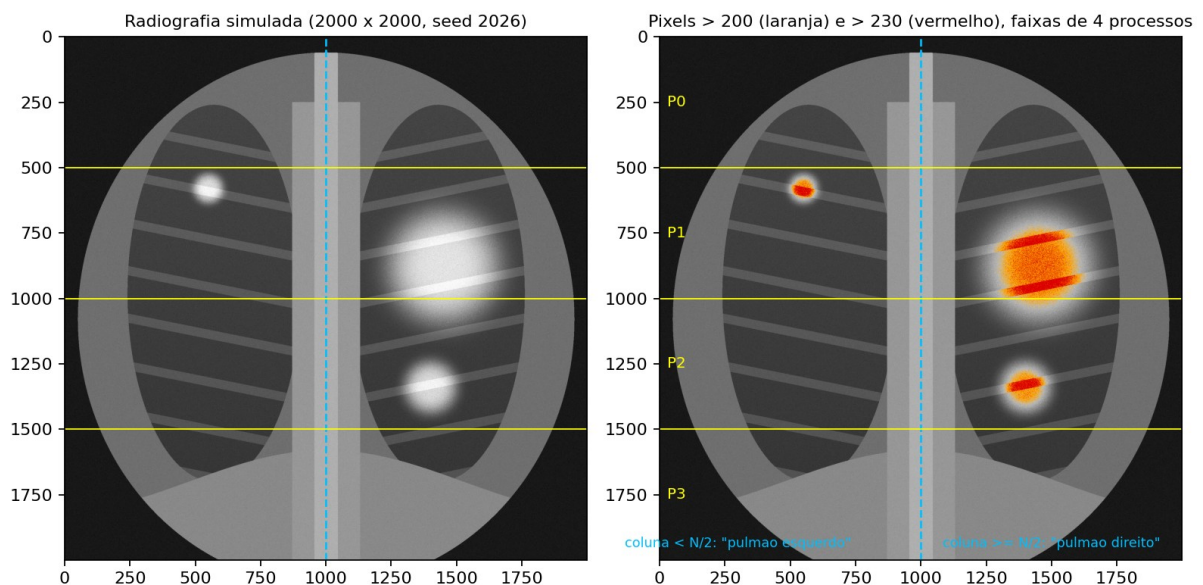


Figura 1: radiografia sintética gerada pelo root (esquerda) e pixels acima dos limiares com as faixas de 4 processos e a divisão de colunas (direita).

Trecho central da distribuição e da análise (arquivo completo no repositório):



```
def dividir_em_faixas(imagem, nprocs):
    blocos = np.array_split(imagem, nprocs, axis=0)      # tamanhos diferem em no maximo 1 linha
    faixas, inicio = [], 0
    for b in blocos:
        faixas.append({"inicio": inicio, "bloco": np.ascontiguousarray(b)})
        inicio += b.shape[0]
    return faixas

parametros = comm.bcast(parametros, root=0)             # Etapa 3
comm.Barrier()                                         # Etapa 4
faixas = dividir_em_faixas(imagem_completa, size) if rank == 0 else None
minha_faixa = comm.scatter(faixas, root=0)             # Etapa 5
...
col_meio = parametros["colunas"] // 2
suspeito_mask = bloco_local > parametros["limiar_suspeito"]
suspeitos_esq = int(np.count_nonzero(suspeito_mask[:, :col_meio])) # coluna < N/2
suspeitos_dir = int(np.count_nonzero(suspeito_mask[:, col_meio:])) # coluna >= N/2
...
comm.Barrier()                                         # Etapa 9
suspeitos_global = comm.reduce(suspeitos_local, op=MPI.SUM, root=0) # Etapa 10
max_global = comm.reduce(max_local, op=MPI.MAX, root=0)
todos_relatorios = comm.gather(relatorio_local, root=0) # Etapa 11
```

2.4 Protocolo experimental

As execuções de demonstração (Seção 3) usam exatamente os comandos do roteiro (docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np P python3 processamento_imagens.py"). As medições de desempenho (Seção 4) foram feitas pelo executar_experimentos.sh, com **3 repetições** por configuração, em três cenários:

- **A (roteiro):** análise vetorizada, latência artificial de 0,5 s x rank nos ranks ímpares, N = 2000 e 4000, com 1, 2, 4 e 8 processos.
- **B (sem latência):** análise vetorizada sem pausas, N = 500, 2000, 4000 e 6000, para isolar o custo real de computação e comunicação.
- **C (carga de CPU alta):** análise pixel a pixel em Python puro (--analise_laco), sem pausas, N = 2000, 4000 e 6000.

O tempo total (T_{total}) é medido com `MPI.Wtime()` do início do programa até o relatório do root, como no template. Também foi registrado o **$T_{paralelo}$** , que exclui a geração da imagem (etapa sequencial que, numa aplicação real, seria a leitura do exame), e a decomposição por fase (geração, scatter, análise, restante). A execução com 1 processo é a linha de base para o speedup $S(p) = T(1)/T(p)$ e a eficiência $E(p) = S(p)/p$. O resultado da análise foi idêntico em todas as configurações (80.446 suspeitos, 4.447 à esquerda e 75.999 à direita para N = 2000), o que confirma que a distribuição não perde nem duplica pixels.



3. Evidências Experimentais (capturas do terminal do GitHub Codespaces)

Todas as capturas abaixo são do terminal integrado do VS Code no GitHub Codespace "verbose tribble" do repositório Artur-Campi/lab-mpi-processamento-imagens (o nome do Codespace aparece na barra de título). As imagens foram recortadas apenas nas bordas vazias; as versões originais em tela cheia estão na pasta prints/ do repositório.

```
@Artur-Campi /workspaces/lab-mpi-processamento-imagens (main) $ docker compose up -d && for n in master worker1 worker2 worker3; do docker compose exec $n service ssh start; docker cp processamento_imagens.py $n:/home/mpiuser/; done; docker compose exec master su - mpiuser -c "cat hosts; mpirun --hostfile hosts -np 4 hostname; python3 -c 'import mpi4py,numpy;print(\"mpi4py\",mpi4py.__version__,\"numpy\",numpy.__version__)'; python3 --version; mpirun --version | head -1"
```

```
[+] up 4/4
✓ Container worker2 Running
✓ Container worker3 Running
✓ Container master Running
✓ Container worker1 Running
* Starting OpenSSH Secure Shell server sshd
Successfully copied 17.8kB (transferred 19.5kB) to master:/home/mpiuser/
* Starting OpenSSH Secure Shell server sshd
Successfully copied 17.8kB (transferred 19.5kB) to worker1:/home/mpiuser/
* Starting OpenSSH Secure Shell server sshd
Successfully copied 17.8kB (transferred 19.5kB) to worker2:/home/mpiuser/
* Starting OpenSSH Secure Shell server sshd
Successfully copied 17.8kB (transferred 19.5kB) to worker3:/home/mpiuser/
```

NAME	IMAGE	CMD	SERVICE	CREATED	STATUS	PORTS
master	lab-mpi:latest	/bin/bash -c 'servi..'	master	4 minutes ago	Up 4 minutes	22/tcp
worker1	lab-mpi:latest	/bin/bash -c 'servi..'	worker1	4 minutes ago	Up 4 minutes	22/tcp
worker2	lab-mpi:latest	/bin/bash -c 'servi..'	worker2	4 minutes ago	Up 4 minutes	22/tcp
worker3	lab-mpi:latest	/bin/bash -c 'servi..'	worker3	4 minutes ago	Up 4 minutes	22/tcp

```
master slots=1
worker1 slots=1
worker2 slots=1
worker3 slots=1
master
worker2
worker1
mpi4py 3.1.3 numpy 1.21.5
Python 3.10.12
mpirun (Open MPI) 4.1.2
```

Figura 2: cluster Docker ativo (4 containers), SSH iniciado, script copiado para os nós, hostfile, teste do mpirun em 4 nós e versões de mpi4py, NumPy, Python e Open MPI.

```
@Artur-Campi /workspaces/lab-mpi-processamento-imagens (main) $ docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 2 python3 processamento_imagens.py" | tee resultados/exec_np2_N2000.txt
[Master] Inicializando analise com 2 processos MPI (2000 x 2000 pixels, cenario 'padrao')...
```

```
=====
RELATORIO CONSOLIDADO DE TRIAGEM DISTRIBUIDA (SIMULACAO)
=====
```

```
--- Informacoes Gerais ---
Dimensoes do Exame      : 2000 x 2000 pixels (4.000.000 px)
Processos MPI Utilizados : 2 (hosts: master, worker1)
Limites Adotados        : suspeito > 200 | alto > 230 | faixa critica >= 5.0%
Latencia Artificial      : 0.5 s x rank (ranks impares)
Tempo Total de Execucao  : 704.64 ms
  geracao(root) 181.3 ms | bcast+barrier 9.1 ms | scatter 2.5 ms | analise(max) 8.2 ms | barreira->fim 0.7 ms
--- Matrices Globais ---
Intensidade Media Global : 87.74 (Maxima: 255)
Total de Pixels Suspeitos : 80446 (2.01%)
Altamente Suspeitos     : 28606 (0.72%)
--- Lateralidade Pulmonar ---
- Pulmao Esquerdo       : 4447 suspeitos
- Pulmao Direito        : 75999 suspeitos
Maior Concentracao      : Pulmao Direito (17.1x o outro lado)
--- Auditoria por Processo (Faixas) ---
P0@master | Linhas [0000-0999] | Susp: 067218 ( 3.36%) | Max: 255 | Espera barreira: 505.3 ms | Faixa: CRITICA
P0@worker1 | Linhas [1000-1999] | Susp: 013228 ( 0.60%) | Max: 255 | Espera barreira: 0.1 ms | Faixa: NORMAL
=====
CLASSIFICACAO GERAL DA RADIOGRAFIA: ATENCAO CLINICA
(faixas criticas: 1/2 | faixas em atencao: 0/2)
=====
```

Figura 3: execução com 2 processos, imagem 2000 x 2000 (comando do roteiro).



```
Problemas Saída Console de Depuração Terminal Portas
@Artur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $ docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 4 python3 processamento_imagens.py" | tee resultados/exec_np4_N2000.txt
[Master] Inicializando análise com 4 processos MPI (2000 x 2000 pixels, cenário 'padrao')...

=====
RELATORIO CONSOLIDADO DE TRIAGEM DISTRIBUIDA (SIMULACAO)
=====
--- Informacoes Gerais ---
Dimensoes do Exame : 2000 x 2000 pixels (4.000.000 px)
Processos MPI Utilizados : 4 (hosts: master, worker1, worker2, worker3)
Limites Adotados : suspeito > 200 | alto > 230 | faixa critica >= 5.0%
Latencia Artificial : 0.5 s x rank (ranks impares)
Tempo Total de Execucão : 1740.02 ms
geracao(root) 214.1 ms | bcst+barrier 15.7 ms | scatter 3.6 ms | analise(max) 4.4 ms | barreira->fim 1.3 ms
--- Metricas Globais ---
Intensidade Media Global : 87.74 (Maxima: 255)
Total de Pixels Suspeitos: 80446 (2.01%)
Altamente Suspeitos : 28606 (0.72%)
--- Lateralidade Pulmonar ---
- Pulmao Esquerdo : 4447 suspeitos
- Pulmao Direito : 75999 suspeitos
Maior Concentracao : Pulmao Direito (17.1x o outro lado)
--- Auditoria por Processo (Faixas) ---
P0@master | Linhas [0000-0099] | Susp: 000000 ( 0.00%) | Max: 200 | Espera barreira: 1502.5 ms | Faixa: NORMAL
P0@worker1 | Linhas [0000-0099] | Susp: 007210 ( 0.72%) | Max: 255 | Espera barreira: 1000.9 ms | Faixa: CRITICA
P0@worker2 | Linhas [1000-1099] | Susp: 013227 ( 1.32%) | Max: 255 | Espera barreira: 1500.6 ms | Faixa: ATENCAO
P0@worker3 | Linhas [1500-1999] | Susp: 000001 ( 0.00%) | Max: 203 | Espera barreira: 0.2 ms | Faixa: NORMAL
=====
CLASSIFICACAO GERAL DA RADIOGRAFIA: ATENCAO CLINICA
(faixas criticas: 1/4 | faixas em atencao: 1/4)
=====

@Artur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $
```

Figura 4: execução com 4 processos, imagem 2000 x 2000, um processo em cada nó.

```
Problemas Saída Console de Depuração Terminal Portas
@Artur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $ docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 8 --oversubscribe python3 processamento_imagens.py" | tee resultados/exec_np8_N2000.txt
[Master] Inicializando análise com 8 processos MPI (2000 x 2000 pixels, cenário 'padrao')...

=====
RELATORIO CONSOLIDADO DE TRIAGEM DISTRIBUIDA (SIMULACAO)
=====
--- Informacoes Gerais ---
Dimensoes do Exame : 2000 x 2000 pixels (4.000.000 px)
Processos MPI Utilizados : 8 (hosts: master, worker1, worker2, worker3)
Limites Adotados : suspeito > 200 | alto > 230 | faixa critica >= 5.0%
Latencia Artificial : 0.5 s x rank (ranks impares)
Tempo Total de Execucão : 4114.56 ms
geracao(root) 523.9 ms | bcst+barrier 30.9 ms | scatter 17.6 ms | analise(max) 8.3 ms | barreira->fim 30.3 ms
--- Metricas Globais ---
Intensidade Media Global : 87.74 (Maxima: 255)
Total de Pixels Suspeitos: 80446 (2.01%)
Altamente Suspeitos : 28606 (0.72%)
--- Lateralidade Pulmonar ---
- Pulmao Esquerdo : 4447 suspeitos
- Pulmao Direito : 75999 suspeitos
Maior Concentracao : Pulmao Direito (17.1x o outro lado)
--- Auditoria por Processo (Faixas) ---
P0@master | Linhas [0000-0249] | Susp: 000000 ( 0.00%) | Max: 199 | Espera barreira: 3510.8 ms | Faixa: NORMAL
P0@master | Linhas [0250-0499] | Susp: 000000 ( 0.00%) | Max: 200 | Espera barreira: 3021.0 ms | Faixa: NORMAL
P0@worker1 | Linhas [0500-0749] | Susp: 003133 ( 1.25%) | Max: 255 | Espera barreira: 3513.5 ms | Faixa: ATENCAO
P0@worker1 | Linhas [0750-0999] | Susp: 001005 (12.22%) | Max: 255 | Espera barreira: 2007.4 ms | Faixa: CRITICA
P0@worker2 | Linhas [1000-1249] | Susp: 001874 ( 0.37%) | Max: 249 | Espera barreira: 3507.8 ms | Faixa: NORMAL
P0@worker2 | Linhas [1250-1499] | Susp: 011353 ( 2.27%) | Max: 255 | Espera barreira: 1000.3 ms | Faixa: ATENCAO
P0@worker3 | Linhas [1500-1749] | Susp: 000001 ( 0.00%) | Max: 203 | Espera barreira: 3506.7 ms | Faixa: NORMAL
P0@worker3 | Linhas [1750-1999] | Susp: 000000 ( 0.00%) | Max: 200 | Espera barreira: 0.1 ms | Faixa: NORMAL
=====
CLASSIFICACAO GERAL DA RADIOGRAFIA: ATENCAO CLINICA
(faixas criticas: 1/8 | faixas em atencao: 2/8)
=====

@Artur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $
```

Figura 5: execução com 8 processos (--oversubscribe), imagem 2000 x 2000. Observe a espera na barreira dos ranks pares (cerca de 3,5 s) contra 0,1 ms do rank 7.

```
Problemas Saída Console de Depuração Terminal Portas
@Artur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $ docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 2 python3 processamento_imagens.py 4000" | tee resultados/exec_np2_N4000.txt
[Master] Inicializando análise com 2 processos MPI (4000 x 4000 pixels, cenário 'padrao')...

=====
RELATORIO CONSOLIDADO DE TRIAGEM DISTRIBUIDA (SIMULACAO)
=====
--- Informacoes Gerais ---
Dimensoes do Exame : 4000 x 4000 pixels (16.000.000 px)
Processos MPI Utilizados : 2 (hosts: master, worker1)
Limites Adotados : suspeito > 200 | alto > 230 | faixa critica >= 5.0%
Latencia Artificial : 0.5 s x rank (ranks impares)
Tempo Total de Execucão : 1263.44 ms
geracao(root) 717.3 ms | bcst+barrier 3.0 ms | scatter 7.8 ms | analise(max) 25.5 ms | barreira->fim 0.8 ms
--- Metricas Globais ---
Intensidade Media Global : 87.77 (Maxima: 255)
Total de Pixels Suspeitos: 32228 (2.01%)
Altamente Suspeitos : 114526 (0.72%)
--- Lateralidade Pulmonar ---
- Pulmao Esquerdo : 17803 suspeitos
- Pulmao Direito : 304425 suspeitos
Maior Concentracao : Pulmao Direito (17.1x o outro lado)
--- Auditoria por Processo (Faixas) ---
P0@master | Linhas [0000-1999] | Susp: 269046 ( 3.36%) | Max: 255 | Espera barreira: 512.9 ms | Faixa: CRITICA
P0@worker1 | Linhas [2000-3999] | Susp: 053182 ( 0.66%) | Max: 255 | Espera barreira: 0.1 ms | Faixa: NORMAL
=====
CLASSIFICACAO GERAL DA RADIOGRAFIA: ATENCAO CLINICA
(faixas criticas: 1/2 | faixas em atencao: 0/2)
=====

@Artur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $
```

Figura 6: execução com 2 processos, imagem 4000 x 4000.



```
Problemas Saída Console de Depuração Terminal Portas
lab-mpi-processamento-imagens [Codigos: verbox: tbbid]

@Bartur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $ docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 4 python3 processamento_imagens.py 4000" | tee resultados/exec_np4_M4000.txt
[Master] Inicializando análise com 4 processos MPI (4000 x 4000 pixels, cenário 'padrao')...

=====
RELATORIO CONSOLIDADO DE TRIAGEM DISTRIBUIDA (SIMULACAO)
=====
--- Informacoes Gerais ---
Dimensoes do Exame : 4000 x 4000 pixels (16.000.000 px)
Processos MPI Utilizados : 4 (hosts: master, worker1, worker2, worker3)
Linhas Adotadas : suspeito > 200 | alto > 230 | faixa critica >= 5.0%
Latencia Artificial : 0.5 s x rank (ranks impares)
Tempo Total de Execucao : 2554.07 ms
  geracao(root) 811.3 ms | broadcast 12.1 ms | scatter 7.3 ms | analise(max) 17.8 ms | barreira->fim 1.3 ms
--- Metricas Globais ---
Intensidade Media Global : 87.77 (Maxima: 255)
Total de Pixels Suspeitos : 22228 (2.01%)
Altamente Suspeitos : 114526 (0.72%)
--- Lateralidade Pulmonar ---
- Pulmao Esquerdo : 17803 suspeitos
- Pulmao Direito : 30425 suspeitos
Maior Concentracao : Pulmao Direito (17.1x o outro lado)
--- Auditoria por Processo (Faixas) ---
P0@master | Linhas [0000-0999] | Susp: 000004 ( 0.00%) | Max: 203 | Espera barreira: 1510.9 ms | Faixa: NORMAL
P0@worker1 | Linhas [1000-1999] | Susp: 269042 ( 6.73%) | Max: 255 | Espera barreira: 1001.3 ms | Faixa: CRITICA
P0@worker2 | Linhas [2000-2999] | Susp: 053180 ( 1.33%) | Max: 255 | Espera barreira: 1500.8 ms | Faixa: ATENCAO
P0@worker3 | Linhas [3000-3999] | Susp: 000002 ( 0.00%) | Max: 204 | Espera barreira: 0.2 ms | Faixa: NORMAL
=====
CLASSIFICACAO GERAL DA RADIOGRAFIA: ATENCAO CLINICA
(faixas criticas: 1/4 | faixas em atencao: 1/4)
=====

@Bartur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $
```

Figura 7: execução com 4 processos, imagem 4000 x 4000.

```
Problemas Saída Console de Depuração Terminal Portas
lab-mpi-processamento-imagens [Codigos: verbox: tbbid]

@Bartur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $ docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 8 --oversubscribe python3 processamento_imagens.py 4000" | tee resultados/exec_np8_M4000.txt
[Master] Inicializando análise com 8 processos MPI (4000 x 4000 pixels, cenário 'padrao')...

=====
RELATORIO CONSOLIDADO DE TRIAGEM DISTRIBUIDA (SIMULACAO)
=====
--- Informacoes Gerais ---
Dimensoes do Exame : 4000 x 4000 pixels (16.000.000 px)
Processos MPI Utilizados : 8 (hosts: master, worker1, worker2, worker3)
Linhas Adotadas : suspeito > 200 | alto > 230 | faixa critica >= 5.0%
Latencia Artificial : 0.5 s x rank (ranks impares)
Tempo Total de Execucao : 4676.78 ms
  geracao(root) 1014.3 ms | broadcast 31.7 ms | scatter 59.2 ms | analise(max) 42.6 ms | barreira->fim 40.2 ms
--- Metricas Globais ---
Intensidade Media Global : 87.77 (Maxima: 255)
Total de Pixels Suspeitos : 22228 (2.01%)
Altamente Suspeitos : 114526 (0.72%)
--- Lateralidade Pulmonar ---
- Pulmao Esquerdo : 17803 suspeitos
- Pulmao Direito : 30425 suspeitos
Maior Concentracao : Pulmao Direito (17.1x o outro lado)
--- Auditoria por Processo (Faixas) ---
P0@master | Linhas [0000-0499] | Susp: 000000 ( 0.00%) | Max: 200 | Espera barreira: 3520.5 ms | Faixa: NORMAL
P0@worker1 | Linhas [0500-0999] | Susp: 000004 ( 0.00%) | Max: 203 | Espera barreira: 3040.9 ms | Faixa: NORMAL
P0@worker1 | Linhas [1000-1499] | Susp: 026458 ( 1.22%) | Max: 255 | Espera barreira: 3512.4 ms | Faixa: ATENCAO
P0@worker1 | Linhas [1500-1999] | Susp: 244584 (12.23%) | Max: 255 | Espera barreira: 2008.9 ms | Faixa: CRITICA
P0@worker2 | Linhas [2000-2499] | Susp: 007566 ( 0.30%) | Max: 246 | Espera barreira: 3408.9 ms | Faixa: NORMAL
P0@worker2 | Linhas [2500-2999] | Susp: 045614 ( 2.20%) | Max: 255 | Espera barreira: 990.7 ms | Faixa: ATENCAO
P0@worker3 | Linhas [3000-3499] | Susp: 000001 ( 0.00%) | Max: 202 | Espera barreira: 3516.0 ms | Faixa: NORMAL
P0@worker3 | Linhas [3500-3999] | Susp: 000001 ( 0.00%) | Max: 204 | Espera barreira: 0.2 ms | Faixa: NORMAL
=====
CLASSIFICACAO GERAL DA RADIOGRAFIA: ATENCAO CLINICA
(faixas criticas: 1/8 | faixas em atencao: 2/8)
=====

@Bartur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $
```

Figura 8: execução com 8 processos, imagem 4000 x 4000.

```
Problemas Saída Console de Depuração Terminal Portas
lab-mpi-processamento-imagens [Codigos: verbox: tbbid]

@Bartur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $ docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 4 python3 processamento_imagens.py --linhas 2003 --colunas 2000" | tee resultados/exec_np4_2003linhas.txt
[Master] Inicializando análise com 4 processos MPI (2003 x 2000 pixels, cenário 'padrao')...

=====
RELATORIO CONSOLIDADO DE TRIAGEM DISTRIBUIDA (SIMULACAO)
=====
--- Informacoes Gerais ---
Dimensoes do Exame : 2003 x 2000 pixels (4.006.000 px)
Processos MPI Utilizados : 4 (hosts: master, worker1, worker2, worker3)
Linhas Adotadas : suspeito > 200 | alto > 230 | faixa critica >= 5.0%
Latencia Artificial : 0.5 s x rank (ranks impares)
Tempo Total de Execucao : 1714.08 ms
  geracao(root) 186.8 ms | broadcast 16.8 ms | scatter 4.7 ms | analise(max) 6.1 ms | barreira->fim 1.2 ms
--- Metricas Globais ---
Intensidade Media Global : 87.74 (Maxima: 255)
Total de Pixels Suspeitos : 80631 (2.01%)
Altamente Suspeitos : 28038 (0.71%)
--- Lateralidade Pulmonar ---
- Pulmao Esquerdo : 4423 suspeitos
- Pulmao Direito : 26208 suspeitos
Maior Concentracao : Pulmao Direito (17.2x o outro lado)
--- Auditoria por Processo (Faixas) ---
P0@master | Linhas [0000-0500] | Susp: 000000 ( 0.00%) | Max: 200 | Espera barreira: 1503.5 ms | Faixa: NORMAL
P0@worker1 | Linhas [0501-1001] | Susp: 007355 ( 0.72%) | Max: 255 | Espera barreira: 1802.2 ms | Faixa: CRITICA
P0@worker2 | Linhas [1002-1502] | Susp: 013275 ( 1.32%) | Max: 255 | Espera barreira: 1694.9 ms | Faixa: ATENCAO
P0@worker3 | Linhas [1503-2002] | Susp: 000001 ( 0.00%) | Max: 203 | Espera barreira: 0.2 ms | Faixa: NORMAL
=====
CLASSIFICACAO GERAL DA RADIOGRAFIA: ATENCAO CLINICA
(faixas criticas: 1/4 | faixas em atencao: 1/4)
=====

@Bartur-Campi →/workspaces/lab-mpi-processamento-imagens (main) $
```

Figura 9: desafio de divisibilidade, 2003 linhas em 4 processos (faixas de 501, 501, 501 e 500 linhas, 4.006.000 pixels contabilizados).



```
Problemas Saída Console de Depuração Terminal Portas
@Artur-Campi → /workspaces/lab-mpi-processamento-imagens (main) $ for p in 1 2 4 8; do docker compose exec -T master su - mpiuser -c "mpirun --hostfile hosts -np $p --oversubscribe python3 processamento_imagens.py 500 --atraso 0 --quiet" | grep -E
I[Tempo Total]geracao"; done | tee resultados/exec_N500_sem_atraso.txt
Processos MPI Utilizados : 1 (hosts: master)
Tempo Total de Execucao : 13.38 ms
  geracao(root) 12.5 ms | bcast+barrier 0.0 ms | scatter 0.2 ms | analise(max) 0.5 ms | barreira->fim 0.1 ms
Processos MPI Utilizados : 2 (hosts: master, worker1)
Tempo Total de Execucao : 22.49 ms
  geracao(root) 17.3 ms | bcast+barrier 3.1 ms | scatter 0.6 ms | analise(max) 0.5 ms | barreira->fim 0.7 ms
Processos MPI Utilizados : 4 (hosts: master, worker1, worker2, worker3)
Tempo Total de Execucao : 41.69 ms
  geracao(root) 18.6 ms | bcast+barrier 20.2 ms | scatter 0.8 ms | analise(max) 0.4 ms | barreira->fim 1.1 ms
Processos MPI Utilizados : 8 (hosts: master, worker1, worker2, worker3)
Tempo Total de Execucao : 111.29 ms
  geracao(root) 32.4 ms | bcast+barrier 23.8 ms | scatter 17.1 ms | analise(max) 0.3 ms | barreira->fim 34.2 ms
@Artur-Campi → /workspaces/lab-mpi-processamento-imagens (main) $ for c in nenhuma padrao extensa; do docker compose exec -T master su - mpiuser -c "mpirun --hostfile hosts -np 4 python3 processamento_imagens.py --cenario $c --atraso 0 --quiet" | grep -E
Io[Pixels Suspeitos]Altamente[Maior[CLASSIFICACAO"]; done | tee resultados/exec_cenarios.txt
[Master] Inicializando analise com 4 processos MPI (2000 x 2000 pixels, cenario 'nenhuma')...
Total de Pixels Suspeitos: 2 (0.00%)
Altamente Suspeitos : 0 (0.00%)
Maior Concentracao : Pulmo Equilibrado (1.0x o outro lado)
CLASSIFICACAO GERAL DA RADIOGRAFIA: SEM INDICIOS RELEVANTES
[Master] Inicializando analise com 4 processos MPI (2000 x 2000 pixels, cenario 'padrao')...
Total de Pixels Suspeitos: 80446 (2.01%)
Altamente Suspeitos : 28000 (0.72%)
Maior Concentracao : Pulmo Direito (17.1x o outro lado)
CLASSIFICACAO GERAL DA RADIOGRAFIA: ATENCAO CLINICA
[Master] Inicializando analise com 4 processos MPI (2000 x 2000 pixels, cenario 'extensa')...
Total de Pixels Suspeitos: 320019 (8.23%)
Altamente Suspeitos : 147086 (3.68%)
Maior Concentracao : Pulmo Direito (1.7x o outro lado)
CLASSIFICACAO GERAL DA RADIOGRAFIA: QUANDO CRITICO / ALTA CONCENTRACAO DE ALTERACOES
@Artur-Campi → /workspaces/lab-mpi-processamento-imagens (main) $
```

Figura 10: imagem reduzida 500 x 500 com 1, 2, 4 e 8 processoss (sem latência) e os três cenários de lesão, que produzem as três classificações gerais.

```
Problemas Saída Console de Depuração Terminal Portas
@Artur-Campi → /workspaces/lab-mpi-processamento-imagens (main) $ head -3 resultados/tempos.csv; wc -l resultados/tempos.csv; sed -n '1,31p' resultados/tabela_desempenho.md
analise_linhas,columns,processos,atraso,t_total_ms,t_paralelo_ms,t_geracao_ms,t_scatter_ms,t_analise_max_ms,suspeitos,esq,dir,diagnostico
vetorizada,2000,2000,1,0.5,143.17,7.77,135.40,0.82,6.74,80446,4447,75999,ATENCAO CLINICA
vetorizada,2000,2000,1,0.5,160.23,7.77,152.46,0.84,6.74,80446,4447,75999,ATENCAO CLINICA
109 resultados/tempos.csv

### Analise vetorizada | latencia artificial = 0.5 s x rank

| N | Proc. | T_total medio (ms) | Desvio | Speedup total | Efic. | T_paralelo (ms) | Speedup paralelo |
|---|---|---|---|---|---|---|---|
| 2000 | 1 | 147.1 | 11.6 | 1.00 | 100% | 7.6 | 1.00 |
| 2000 | 2 | 701.9 | 6.8 | 0.21 | 10% | 521.4 | 0.01 |
| 2000 | 4 | 1719.4 | 18.3 | 0.09 | 2% | 1537.5 | 0.00 |
| 2000 | 8 | 4047.4 | 45.0 | 0.04 | 0% | 3632.8 | 0.00 |
| 4000 | 1 | 617.2 | 78.2 | 1.00 | 100% | 31.1 | 1.00 |
| 4000 | 2 | 1248.3 | 12.6 | 0.49 | 25% | 551.1 | 0.06 |
| 4000 | 4 | 2316.4 | 71.7 | 0.27 | 7% | 1556.1 | 0.02 |
| 4000 | 8 | 5430.1 | 17.9 | 0.11 | 1% | 3646.7 | 0.01 |

### Analise vetorizada | latencia artificial = 0.0 s x rank

| N | Proc. | T_total medio (ms) | Desvio | Speedup total | Efic. | T_paralelo (ms) | Speedup paralelo |
|---|---|---|---|---|---|---|---|
| 500 | 1 | 13.2 | 0.0 | 1.00 | 100% | 0.7 | 1.00 |
| 500 | 2 | 22.3 | 0.1 | 0.59 | 30% | 5.7 | 0.13 |
| 500 | 4 | 60.9 | 24.1 | 0.22 | 5% | 34.3 | 0.02 |
| 500 | 8 | 128.7 | 25.4 | 0.10 | 1% | 94.8 | 0.01 |
| 2000 | 1 | 144.1 | 5.2 | 1.00 | 100% | 8.1 | 1.00 |
| 2000 | 2 | 201.6 | 6.3 | 0.71 | 36% | 21.6 | 0.38 |
| 2000 | 4 | 263.6 | 90.2 | 0.55 | 14% | 36.3 | 0.22 |
| 2000 | 8 | 510.5 | 43.9 | 0.28 | 4% | 136.8 | 0.06 |
| 4000 | 1 | 569.8 | 5.0 | 1.00 | 100% | 30.6 | 1.00 |
| 4000 | 2 | 729.2 | 7.0 | 0.78 | 35% | 49.6 | 0.62 |
| 4000 | 4 | 884.3 | 140.7 | 0.64 | 16% | 60.3 | 0.51 |
| 4000 | 8 | 1337.4 | 364.4 | 0.43 | 5% | 162.9 | 0.19 |
| 6000 | 1 | 1503.9 | 101.9 | 1.00 | 100% | 142.6 | 1.00 |
@Artur-Campi → /workspaces/lab-mpi-processamento-imagens (main) $
```

Figura 11: conclusão da bateria de 108 medições e tabela gerada no Codespace (parte 1).



```
Problemas Saída Console de Depuração Terminal Portas
@Artur-Campi → /workspaces/lab-mpi-processamento-imagens (main) $ sed -n '32,$p' resultados/tabela_desempenho.md; ls -la resultados/
| 6000 | 2 | 1733.3 | 89.5 | 0.87 | 43% | 147.3 | 0.97 |
| 6000 | 4 | 1793.2 | 119.2 | 0.84 | 21% | 139.6 | 1.02 |
| 6000 | 8 | 3240.1 | 231.9 | 0.46 | 6% | 287.6 | 0.50 |

### Analise laco | latencia artificial = 0.0 s x rank

| N | Proc. | T_total medio (ms) | Desvio | Speedup total | Efic. | T_paralelo (ms) | Speedup paralelo |
|---|---|---|---|---|---|---|---|
| 2000 | 1 | 486.3 | 17.1 | 1.00 | 100% | 354.7 | 1.00 |
| 2000 | 2 | 496.2 | 28.8 | 0.98 | 49% | 301.1 | 1.18 |
| 2000 | 4 | 403.9 | 87.3 | 1.20 | 30% | 218.0 | 1.63 |
| 2000 | 8 | 687.7 | 56.0 | 0.71 | 9% | 351.0 | 1.01 |
| 4000 | 1 | 2012.0 | 102.7 | 1.00 | 100% | 1476.3 | 1.00 |
| 4000 | 2 | 2010.9 | 58.5 | 1.00 | 50% | 1290.3 | 1.14 |
| 4000 | 4 | 1442.5 | 76.4 | 1.39 | 35% | 679.3 | 2.17 |
| 4000 | 8 | 2230.4 | 292.4 | 0.90 | 11% | 798.6 | 1.85 |
| 6000 | 1 | 4710.1 | 17.3 | 1.00 | 100% | 3552.6 | 1.00 |
| 6000 | 2 | 4499.7 | 76.5 | 1.05 | 52% | 2935.3 | 1.21 |
| 6000 | 4 | 3491.6 | 14.3 | 1.35 | 34% | 1721.7 | 2.06 |
| 6000 | 8 | 5319.1 | 161.5 | 0.89 | 11% | 2326.0 | 1.53 |

Gráfico salvo em resultados/grafico_speedup.png
total 232
drwxrwxrwt 2 codespace codespace 4096 Sep 28 01:58
drwxrwxrwt 5 codespace root 4096 Sep 28 01:46
-rw-rw-rw- 1 codespace codespace 682 Sep 28 01:28 ambiente_codespace.txt
-rw-rw-rw- 1 codespace codespace 832 Sep 28 01:37 exec_N500_sem_atraso.txt
-rw-rw-rw- 1 codespace codespace 909 Sep 28 01:37 exec_cenarios.txt
-rw-rw-rw- 1 codespace codespace 1578 Sep 28 01:34 exec_np2_N2000.txt
-rw-rw-rw- 1 codespace codespace 1585 Sep 28 01:35 exec_np2_N4000.txt
-rw-rw-rw- 1 codespace codespace 1827 Sep 28 01:36 exec_np4_2003linhas.txt
-rw-rw-rw- 1 codespace codespace 1827 Sep 28 01:35 exec_np4_N2000.txt
-rw-rw-rw- 1 codespace codespace 1833 Sep 28 01:36 exec_np4_N4000.txt
-rw-rw-rw- 1 codespace codespace 2286 Sep 28 01:35 exec_np8_N2000.txt
-rw-rw-rw- 1 codespace codespace 2293 Sep 28 01:36 exec_np8_N4000.txt
-rw-rw-rw- 1 codespace codespace 94316 Sep 28 01:50 grafico_speedup.png
-rw-rw-rw- 1 codespace codespace 71465 Sep 28 01:29 log_preparar_cluster.txt
-rw-rw-rw- 1 codespace codespace 2667 Sep 28 01:50 tabela_desempenho.md
-rw-rw-rw- 1 codespace codespace 10102 Sep 28 01:50 tempos.csv
@Artur-Campi → /workspaces/lab-mpi-processamento-imagens (main) $
```

Figura 12: tabela gerada no Codespace (parte 2) e arquivos de resultados produzidos.

4. Tabela Comparativa de Desempenho e Speedup

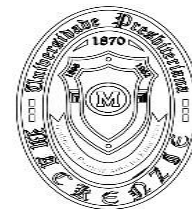
Médias de 3 repetições. Speedup e eficiência calculados em relação à execução com 1 processo no mesmo cenário.

4.1 Cenário A: configuração do roteiro (latência artificial de 0,5 s x rank)

N	Processos	T_total (ms)	Desvio (ms)	Speedup	Eficiência	Pausa do rank mais lento (ms)
2000	1	147,1	11,6	1,00	100%	0
2000	2	701,9	6,8	0,21	10%	500
2000	4	1.719,4	18,3	0,09	2%	1.500
2000	8	4.047,4	45,0	0,04	0%	3.500
4000	1	617,2	78,2	1,00	100%	0
4000	2	1.248,3	12,6	0,49	25%	500
4000	4	2.316,4	71,7	0,27	7%	1.500
4000	8	5.430,1	17,9	0,11	1%	3.500

4.2 Cenário B: análise vetorizada sem latência artificial

N	Proc.	T_total (ms)	Speedup	Efic.	Geração (ms)	Scatter (ms)	Análise máx. (ms)	Speedup T_paralelo
500	1	13,2	1,00	100%	12,5	0,1	0,5	1,00
500	2	22,3	0,59	30%	16,6	0,5	0,5	0,13



500	4	60,9	0,22	5%	26,5	0,7	0,4	0,02
500	8	128,7	0,10	1%	33,9	15,7	0,3	0,01
2000	1	144,1	1,00	100%	136,0	0,8	7,1	1,00
2000	2	201,6	0,71	36%	180,0	2,0	8,2	0,38
2000	4	263,6	0,55	14%	227,3	4,5	6,2	0,22
2000	8	510,5	0,28	4%	373,6	35,5	12,2	0,06
4000	1	569,8	1,00	100%	539,2	2,9	27,5	1,00
4000	2	729,2	0,78	39%	679,6	9,9	25,4	0,62
4000	4	884,3	0,64	16%	824,0	9,1	31,5	0,51
4000	8	1.337,4	0,43	5%	1.174,5	43,8	32,9	0,19
6000	1	1.503,9	1,00	100%	1.361,3	71,7	70,7	1,00
6000	2	1.733,3	0,87	43%	1.586,0	73,9	53,8	0,97
6000	4	1.793,2	0,84	21%	1.653,6	82,1	40,5	1,02
6000	8	3.240,1	0,46	6%	2.952,4	170,6	48,0	0,50

4.3 Cenário C: análise pixel a pixel em Python puro (carga de CPU alta)

N	Proc.	T_total (ms)	Speedup total	Efic.	T_paralelo (ms)	Speedup T_paralelo	Análise máx. (ms)	Speedup análise
2000	1	486,3	1,00	100%	354,7	1,00	353,7	1,00
2000	2	496,2	0,98	49%	301,1	1,18	292,9	1,21
2000	4	403,9	1,20	30%	218,0	1,63	185,2	1,91
2000	8	687,7	0,71	9%	351,0	1,01	212,4	1,67
4000	1	2.012,0	1,00	100%	1.476,3	1,00	1.473,0	1,00
4000	2	2.010,9	1,00	50%	1.290,3	1,14	1.263,5	1,17
4000	4	1.442,5	1,39	35%	679,3	2,17	652,7	2,26
4000	8	2.230,4	0,90	11%	798,6	1,85	697,9	2,11
6000	1	4.710,1	1,00	100%	3.552,6	1,00	3.474,7	1,00
6000	2	4.499,7	1,05	52%	2.935,3	1,21	2.843,3	1,22
6000	4	3.491,6	1,35	34%	1.721,7	2,06	1.609,4	2,16
6000	8	5.319,1	0,89	11%	2.326,0	1,53	2.067,4	1,68

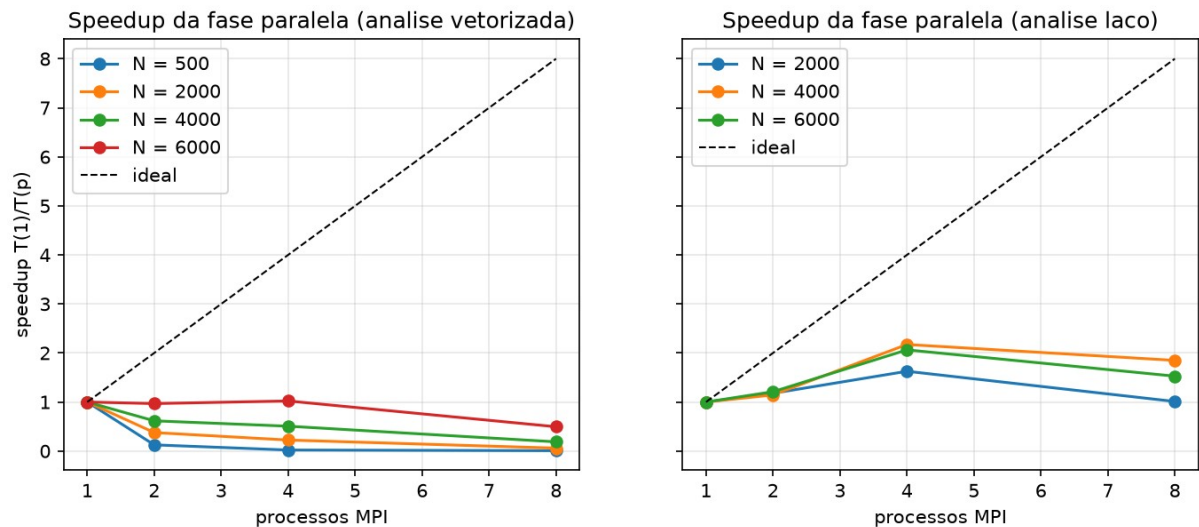


Figura 13: speedup da fase paralela (T_{paralelo}) gerado pelo analisar_resultados.py no Codespace. Linha tracejada: speedup ideal linear.

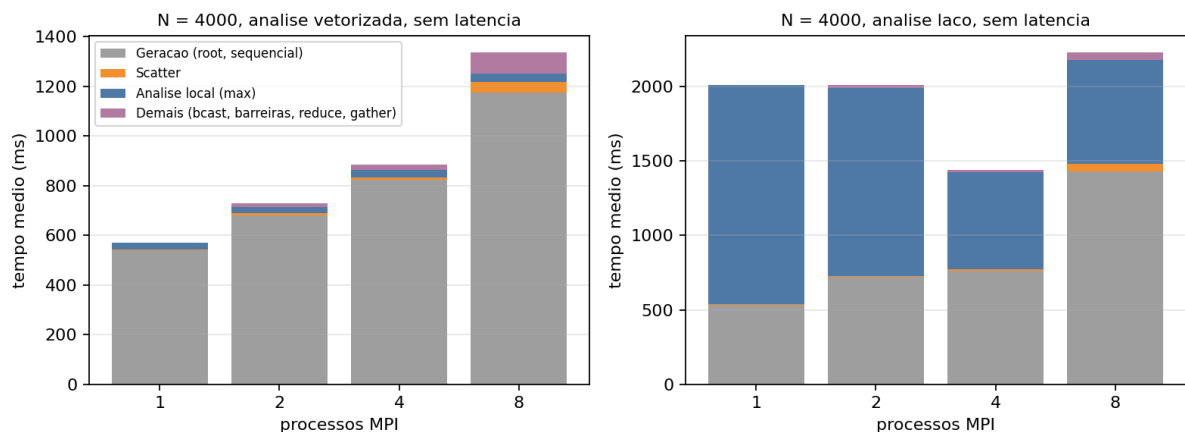


Figura 14: decomposição do tempo médio por fase para $N = 4000$, sem latência artificial (dados de resultados/tempos.csv).

4.4 Análise da eficiência paralela

Com a análise vetorizada, o NumPy percorre os 16 milhões de pixels de uma imagem 4000 x 4000 em cerca de 27 ms, enquanto a geração sequencial da imagem no root leva 539 ms. A fração sequencial é portanto $f = 0,95$ e a Lei de Amdahl limita o speedup a $1/f = 1,06$, mesmo com infinitos processos. Na prática o tempo **aumenta** com mais processos, porque o custo de serializar e distribuir as faixas e de sincronizar supera o pouco que se economiza na análise. Com 8 processos o efeito piora: há 8 processos em 4 vCPUs (2 núcleos físicos), e os processos que aguardam em uma coletiva do Open MPI fazem espera ativa (polling), disputando CPU com o root; por isso até a geração fica mais lenta (539 ms com 1 processo contra 1.174 ms com 8).

No cenário C, o trabalho por pixel é muito maior e a fração sequencial cai para $f = 0,27$ ($N = 4000$). Aí a distribuição compensa: com 4 processos a análise é 2,26x mais rápida e o tempo total cai de 2.012 ms para 1.443 ms (speedup 1,39). Pela Lei de Amdahl, o máximo esperado com 4 processos seria $1/(f + (1-f)/4) = 2,22$; o valor medido fica abaixo porque os 4 vCPUs são 2 núcleos físicos com SMT (duas threads no mesmo núcleo não rendem o dobro) e porque ainda há overhead de scatter e sincronização. Com 2 processos o ganho é pequeno (1,14 a 1,21 na fase paralela), compatível com os dois processos caindo em threads irmãs do mesmo núcleo físico. Com 8 processos o speedup volta a cair: há mais processos que vCPUs.



5. Respostas às 9 Questões de Reflexão

Questão 1: Impacto do número de processos no tempo de execução

Não. Em nenhum cenário o tempo diminuiu de forma linear. No cenário do roteiro (Tabela 4.1) o tempo total **cresceu** com os processos (N = 2000: 147 ms com 1 processo, 702 ms com 2, 1.719 ms com 4 e 4.047 ms com 8), porque a pausa artificial do rank ímpar mais alto (0,5, 1,5 e 3,5 s) domina tudo. Mesmo sem as pausas (cenário B) o tempo aumentou: a análise vetorizada é tão barata que a fração sequencial (geração da imagem no root) chega a 95% do tempo com 1 processo, e a Lei de Amdahl limita o ganho a 1,06x. Sobre essa parte fixa ainda se somam os custos de serializar cada faixa com pickle (o `comm.scatter` minúsculo copia os arrays), de transmiti-la pela rede virtual do Docker e das sincronizações, que crescem com o número de processos. Só quando o trabalho por pixel ficou pesado (cenário C) houve ganho real, de até 1,39x no total e 2,26x na análise com 4 processos, ainda limitado pelos 2 núcleos físicos do Codespace e pela fração sequencial de cerca de 27%.

Questão 2: Influência de processos lentos (stragglers)

O tempo global passou a ser praticamente a pausa do processo mais lento somada ao restante do programa. Com 2 processos o rank 1 dorme 0,5 s e o T_{paralelo} médio foi 521 ms; com 4 processos o rank 3 dorme 1,5 s e o T_{paralelo} foi 1.538 ms; com 8 processos o rank 7 dorme 3,5 s e o T_{paralelo} foi 3.633 ms. A auditoria da Figura 5 mostra o fenômeno por dentro: os ranks pares, sem pausa, esperaram cerca de 3,5 s na barreira, o rank 5 esperou 1,0 s, o rank 3 esperou 2,0 s e o rank 7 esperou só 0,1 ms, porque foi o último a chegar. Em uma aplicação síncrona, a barreira e as coletivas bloqueantes (`reduce` e `gather`) só se completam quando todos os participantes contribuem, então a fase inteira termina no instante em que o elo mais lento termina. Todo o tempo que os nós rápidos economizaram vira espera ociosa. Por isso, em clusters heterogêneos, divisões estáticas e iguais de trabalho desperdiçam capacidade, e técnicas como balanceamento dinâmico (fila de tarefas) ou divisão proporcional à velocidade de cada nó são preferidas.

Questão 3: Papel prático do MPI_Barrier

A primeira barreira (Etapa 4) garante que nenhum processo comece o tráfego pesado do `scatter` antes de todo o cluster ter recebido e aplicado os parâmetros; ela também delimita com precisão o início da fase de distribuição nas medições. A segunda barreira (Etapa 9) foi a mais relevante na prática: ela separa a fase de processamento da consolidação e torna explícito e mensurável o straggler effect (o campo "Espera barreira" da auditoria só existe por causa dela). Vale notar que, do ponto de vista da correção, a barreira não é obrigatória neste programa, porque `bcast`, `scatter`, `reduce` e `gather` já são coletivas bloqueantes: sem ela, o `reduce` continuaria esperando o processo mais lento. Enquanto aguardam, os nós rápidos ficam parados sem trabalho útil. No Open MPI essa espera é ativa (`busy polling`), então eles ainda consomem CPU; isso ficou visível com 8 processos em 4 vCPUs, quando até a geração da imagem no root ficou mais lenta.

Questão 4: Broadcast vs. Scatter

Os parâmetros são poucos bytes (um dicionário com dimensões, limiares e percentuais) e todos os processos precisam de todos eles, então o `broadcast` é a operação natural. A imagem, ao contrário, é grande e cada processo só precisa da sua faixa: o `scatter` entrega a cada um apenas $1/\text{size}$ dos dados. Para N = 4000 a imagem tem 16 MB (uint8). Com `scatter` e 8 processos, cada um recebe cerca de 2 MB e o total trafegado é de cerca de 16 MB. Com `broadcast` da imagem inteira, cada um dos 8 processos receberia e guardaria 16 MB, totalizando 128 MB de tráfego e 8 cópias completas em memória (com pickle, ainda há a cópia serializada temporária). O tempo de comunicação e o consumo de memória cresceriam proporcionalmente ao número de processos, e cada processo ainda teria de saber recortar a sua parte, desperdiçando rede e RAM. Para uma



imagem 6000 x 6000 seriam 36 MB por nó; em um cluster real com rede física, isso rapidamente se tornaria o gargalo.

Questão 5: Diferença prática entre Reduce e Gather

Os dois mecanismos resolvem necessidades diferentes. Informações que só interessam agregadas foram consolidadas com **reduce**: total de pixels, soma das intensidades (para a média global), total de suspeitos e altamente suspeitos, suspeitos por pulmão (todos com MPI.SUM) e intensidade máxima global (MPI.MAX). Essas operações são associativas, o resultado tem tamanho constante e o root não precisa fazer contas. Já a auditoria exige **preservar a individualidade** de cada faixa: rank, host, intervalo de linhas, número de suspeitos, máximo local, classificação e tempo de espera na barreira. Somar esses dados destruiria a informação, então eles foram coletados com **gather**, que devolve uma lista ordenada por rank. Usar só gather funcionaria, mas transferiria mais dados e obrigaria o root a fazer toda a agregação; usar só reduce perderia a visão por faixa, que é justamente o que permite localizar a lesão (ex.: P03 com 12,22% de suspeitos na Figura 5).

Questão 6: Escalabilidade em imagens pequenas

Não é vantajoso. Para 500 x 500 (250 mil pixels), o tempo total foi 13,2 ms com 1 processo, 22,3 ms com 2, 60,9 ms com 4 e 128,7 ms com 8 (Figura 10 e Tabela 4.2). A análise local leva menos de 1 ms, enquanto só a distribuição, as barreiras e a consolidação custam dezenas de milissegundos. A granularidade (computação por unidade de comunicação) é baixíssima, então quanto mais processos, pior. Além disso, esses números não incluem o custo de o mpirun lançar processos via SSH em outros containers, que é da ordem de centenas de milissegundos. Distribuir só compensa quando cada processo recebe trabalho suficiente para amortizar esses custos fixos, como no cenário C com imagens de 4000 x 4000 ou maiores.

Questão 7: Detecção de lateralidade anatômica

A imagem foi particionada **apenas no eixo das linhas** (`np.array_split` com `axis=0`). Assim, toda faixa recebida por um processo contém todas as N colunas, com os mesmos índices de coluna da imagem original. Isso permite que cada processo classifique a lateralidade localmente, sem nenhum deslocamento: o pixel da coluna j pertence ao pulmão esquerdo se $j < \text{floor}(N/2)$ e ao direito se $j \geq \text{floor}(N/2)$ (`fatias mascara[:, :col_meio]` e `mascara[:, col_meio:]`). Somando essas contagens com **reduce**, o root obtém a lateralidade global: 4.447 suspeitos no lado esquerdo contra 75.999 no direito (17,1 vezes mais), o que corresponde exatamente aos focos injetados (dois no lado direito e um pequeno no esquerdo). A informação geométrica que garante a precisão é a preservação da coordenada de coluna, que o particionamento horizontal não altera. Uma ressalva: seguimos a convenção do roteiro (lado esquerdo da matriz = pulmão esquerdo). Em uma radiografia PA real, o lado direito do paciente aparece à esquerda de quem olha a imagem, então um sistema clínico precisaria considerar a orientação do exame.

Questão 8: Balanceamento de carga

Em volume de dados, a distribuição foi uniforme: `np.array_split` entrega o mesmo número de linhas a todos (diferença máxima de 1 linha) e, como a análise vetorizada custa o mesmo por pixel independentemente do valor, os tempos de análise por processo foram equivalentes. Porém, a distribuição das **suspeitas** foi muito desigual. Com 8 processos e $N = 2000$ (Figura 5), as faixas apicais P00 e P01 tiveram 0 suspeitos, as basais P06 e P07 tiveram 1 e 0, enquanto as faixas medianas concentraram quase tudo: P03 (linhas 750 a 999) sozinha teve 61.085 suspeitos, 76% do total, seguida de P05 com 11.353 e P02 com 6.133. Isso reflete a posição dos focos de consolidação no terço médio e na base do pulmão direito. Neste problema, esse desequilíbrio de conteúdo quase não afeta o tempo (no modo laço em Python há um pequeno custo extra nos pixels suspeitos, por causa dos ramos condicionais). O desequilíbrio de tempo realmente relevante veio da



heterogeneidade simulada de hardware (Etapa 8). Em uma análise mais cara por pixel suspeito (por exemplo, segmentação detalhada da lesão), seria necessário um balanceamento dinâmico.

Questão 9: Desafio de divisibilidade de linhas

Com 2003 linhas e 4 processos, $2003 = 4 \times 500 + 3$. O template original usava `np.split`, que exige divisão exata e lançaria `ValueError`. A solução adotada foi o **fatiamiento balanceado** com `np.array_split`: as 3 primeiras faixas recebem 501 linhas e a última 500 (resto distribuído uma linha por processo). A Figura 9 comprova a execução: faixas [0000-0500], [0501-1001], [1002-1502] e [1503-2002], 4.006.000 pixels contabilizados no `reduce` e nenhuma falha. Como cada faixa viaja junto com o índice global da sua primeira linha, a auditoria continua exata. Alternativas válidas: (a) **padding** com linhas nulas até o próximo múltiplo de `size` (valores 0 não passam dos limiares, mas os pixels artificiais precisariam ser descontados do total e da média); (b) usar **Scatterv** (versão com buffers do MPI) informando contagens e deslocamentos diferentes por processo, evitando o `pickle`; (c) o root processar sozinho as linhas restantes; (d) redimensionar a imagem, que descartaria ou inventaria informação e por isso é a pior opção para um exame médico.

6. Dificuldades Técnicas e Soluções

mpirun e ssh travando no Codespaces. Ao executar o primeiro `mpirun`, o comando ficou parado. O diagnóstico com `ssh -v` mostrou que a conexão TCP de master para `worker1` não se completava, e até `ping` entre containers falhava (100% de perda). A causa: no Codespaces, as tabelas `iptables-legacy` do host estão com política `FORWARD DROP` e só liberam a `docker0`, então o tráfego da rede customizada `mpinet` era descartado. Solução: `sudo iptables-legacy -I DOCKER-USER -j ACCEPT`, já incorporada ao `preparar_cluster.sh` (o log resultados/log_preparar_cluster.txt registra a tentativa que travou).

Chaves SSH entre os nós. Em vez de copiar chaves manualmente, a chave `ed25519` do `mpiuser` é gerada no build da imagem e colocada no `authorized_keys`. Como os 4 containers usam a mesma imagem, todos confiam uns nos outros. O `~/.ssh/config` desativa a checagem de host (`StrictHostKeyChecking no`), evitando a pergunta interativa que travaria o `mpirun`.

Open MPI dentro de containers. O transporte de memória compartilhada (`vader`) tenta usar `CMA/ptrace`, bloqueado por padrão no Docker, o que gera erros de leitura. Foi configurado `btl_vader_single_copy_mechanism = none` e as interfaces TCP foram restringidas a `eth0` no arquivo de parâmetros MCA da imagem. Como `su - mpiuser` limpa as variáveis de ambiente, a configuração precisou ficar em arquivo e não em `ENV`.

Fatiamiento com np.split. O `np.split` do template falha quando as linhas não são múltiplas de `size`. Foi trocado por `np.array_split` e cada bloco passou a levar o índice global da primeira linha, porque a fórmula `rank * linhas_por_proc` do template ficaria errada com faixas de tamanhos diferentes.

Tipos de dados. Somar um bloco `uint8` poderia estourar em implementações ingênuas; a soma é feita com `sum(dtype=np.uint64)`. Todos os valores enviados ao `reduce` e ao `gather` são convertidos para `int` do Python, evitando misturar tipos NumPy na serialização e garantindo que `MPI.SUM` e `MPI.MAX` operem sobre inteiros comuns.

Speedup menor que 1. A primeira bateria mostrou que mais processos sempre deixavam o programa mais lento. Em vez de mascarar o resultado, a decomposição por fase (geração, scatter, análise) mostrou que a análise NumPy era rápida demais e a geração sequencial dominava. Foi adicionado o modo `--analise laco` (inspeção literal pixel a pixel em Python) para medir um cenário com granularidade maior, em que o paralelismo compensa, e os dois cenários foram mantidos no relatório.



Hardware compartilhado. O Codespace de "4 núcleos" tem 2 núcleos físicos com SMT. Isso explica o speedup limitado a cerca de 2 e o pior desempenho com 8 processos (oversubscribe), e foi levado em conta na interpretação.

7. Declaração e Análise do Uso de IA

7.1 Registro explícito

Este laboratório foi desenvolvido com uso intensivo de IA generativa: o **Claude (Anthropic)**, no modo Cowork do aplicativo Claude, com acesso ao navegador. Nenhuma outra ferramenta de IA (ChatGPT, Gemini, Copilot etc.) foi usada. O Claude foi empregado em:

- Leitura do enunciado em PDF e planejamento da solução.
- Escrita do `processamento_imagens.py` (a partir do template da atividade), do Dockerfile, do `docker-compose.yml`, do hostfile e dos scripts de experimento e análise.
- Teste local do código com Open MPI antes de ir para o Codespaces.
- Criação do repositório no GitHub, envio dos arquivos, criação do Codespace de 4 núcleos e execução de todos os comandos no terminal do Codespace por automação do navegador.
- Diagnóstico do travamento do SSH/MPI (iptables) e captura das telas do terminal.
- Redação deste relatório a partir dos resultados medidos.

7.2 Principais solicitações (prompts)

- "Faça o que se pede [no PDF do laboratório], mas dessa vez quero que você crie um repositório no GitHub para esse laboratório, suba os arquivos, e os prints precisam ser do terminal real do GitHub Codespace."
- Escolhas feitas durante a execução: repositório público e Codespace de 4 núcleos.
- Continuação após o login no GitHub ("loguei").

7.3 Análise crítica obrigatória

Veracidade. Todos os números deste relatório vêm dos arquivos gerados no Codespace (`resultados/tempos.csv` e logs), e as capturas são do terminal real, como pedido. As explicações teóricas (Amdahl, granularidade, stragglers, diferença entre `reduce` e `gather`) foram conferidas com a documentação do MPI e do `mpi4py` e são coerentes com os dados. Mesmo assim, a IA errou ou precisou de correção em alguns pontos, o que reforça que a saída dela precisa de verificação:

- A primeira versão do cluster travou no Codespaces: a configuração que funcionou no teste local não funcionava lá. Foi preciso depurar a rede (`ssh -v`, `ping`, `iptables`) para achar a causa.
- A hipótese inicial de que o speedup seria visível com a análise vetorizada estava errada; os dados mostraram o contrário e exigiram um cenário extra.
- O template da atividade tinha limitações (`np.split` sem tratar resto, cálculo de `linhas_inicio` fixo) que precisaram de correção.

Aprendizagem. O ponto mais importante do laboratório foi perceber, com dados reais, que "mais processos" não significa "mais rápido": o ganho depende da fração sequencial, do custo de comunicação em relação ao cálculo e do hardware físico disponível. Também ficou claro o papel de cada coletiva (`bcast` para dados pequenos e globais, `scatter` para dados grandes e particionáveis, `reduce` para agregados, `gather` para dados individuais) e o custo prático de sincronização em sistemas com nós desiguais. Como o código e o texto foram



produzidos com apoio de IA, o aluno revisou os resultados e as respostas e é responsável pelo conteúdo entregue.

8. Conclusão e Referências

O sistema distribuído implementou as 12 etapas pedidas e usou os cinco mecanismos coletivos exigidos (Bcast, Scatter, Barrier, Reduce e Gather) em um cluster Docker de 4 nós no GitHub Codespaces, com execuções comprovadas para 2, 4 e 8 processos e imagens de 2000 x 2000 e 4000 x 4000. O diagnóstico simulado foi consistente em todas as configurações (Atenção Clínica, com maior concentração no pulmão direito), e o fatiamento balanceado resolveu o caso de 2003 linhas sem perda de dados.

Os experimentos mostraram que o desempenho de uma aplicação MPI é determinado por fatores além do número de processos: com a análise vetorizada, a fração sequencial e o overhead de comunicação fizeram o tempo crescer com mais processos; com uma análise mais pesada por pixel, houve speedup de até 2,3x na análise com 4 processos, perto do limite imposto pelos 2 núcleos físicos da máquina. A latência artificial mostrou na prática como um único nó lento dita o tempo de todo o cluster síncrono.

Referências

MESSAGE PASSING INTERFACE FORUM. MPI: A Message-Passing Interface Standard, Version 4.1. 2023. Disponível em: <https://www.mpi-forum.org/docs/>.

DALCIN, L.; FANG, Y.-L. L. mpi4py: Status Update After 12 Years of Development. Computing in Science & Engineering, v. 23, n. 4, p. 47-54, 2021.

MPI4PY. MPI for Python: documentação oficial. Disponível em: <https://mpi4py.readthedocs.io/>.

OPEN MPI PROJECT. Open MPI v4.1 documentation (mpirun, hostfile, parâmetros MCA). Disponível em: <https://www.open-mpi.org/doc/v4.1/>.

HARRIS, C. R. et al. Array programming with NumPy. Nature, v. 585, p. 357-362, 2020.

AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. AFIPS Conference Proceedings, v. 30, p. 483-485, 1967.

GITHUB. GitHub Codespaces documentation. Disponível em: <https://docs.github.com/codespaces>.

DOCKER. Docker Compose documentation. Disponível em: <https://docs.docker.com/compose/>.

UNIVERSIDADE PRESBITERIANA MACKENZIE. Atividade Lab: Processamento Distribuído de Imagens Médicas com MPI (MPI + Python + Docker). Prof. Alcides e Prof. Mário Menezes. Material da disciplina Computação Distribuída, 2026.